Here's the complete file for copy/paste:

WCEPS Email Generator - Cursor Setup Guide

Project Overview

Florida and all WIDA states eventually District Email System - AI-powered personalized sales email generation for 67 Florida school districts. The system:

1. **Scrapes district websites** via Firecrawl (strategic plans, leadership, ESOL programs)
2. **Extracts contacts** with confidence scoring and Perplexity verification
3. **Researches funding/compliance** via Perplexity sonar-pro
4. **Generates personalized emails** with GPT-5 using district context
5. **Exports to Salesforce** Account Engagement format

Quick Start

1. Clone & Install
```bash
git clone
cd
npm install
npm run dev
```

2. Environment Variables
Create a `.env` file:
```env
VITE_SUPABASE_URL=https://dkkdckoihsylkaldqger.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6ImRra2Rja29paHN5bGthbGRxZ2VyIiwicm9sZSI6ImFub24iLCJpYXQiOjE3NjM3NTQ5MjksImV4cCI6MjA3OTMzMDkyOX0.NlWz8HoR4ZkX8VO1eXs2ZYgQnTSyoPk-63SoApmCDWM
VITE_SUPABASE_PROJECT_ID=dkkdckoihsylkaldqger
```

3. Required Secrets (Edge Functions)

Secret	Purpose

`OPENAI_API_KEY`	Reranking, contact extraction, email composition
`LOVABLE_API_KEY`	AI Gateway for GPT-5 email generation
`FIRECRAWL_API_KEY`	Website scraping & sitemap mapping
`PERPLEXITY_API_KEY`	Funding research, compliance research, contact verification

---

Technology Stack

Layer	Technology
Frontend	React 18 + TypeScript + Vite
Styling	Tailwind CSS + shadcn/ui
State	TanStack Query
Routing	React Router DOM v6
Backend	Supabase Edge Functions (Deno)
Database	PostgreSQL
AI Models	GPT-5 (emails), GPT-4o (reranking), Perplexity sonar-pro

---

Project Structure (Email-Focused)

```
├── src/
│   ├── components/
│   │   └── districts/
│   │       └── DistrictDetailDialog.tsx  # District info modal
│   │   └── ui/                          # shadcn components
│   │
│   ├── pages/
│   │   ├── DistrictEmails.tsx           # Main bulk email UI
│   │   ├── EmailGenerator.tsx           # Single email generator
│   │   ├── Auth.tsx                     # Login/signup
│   │   └── Index.tsx                    # Landing
│   │
│   ├── lib/api/
│   │   └── firecrawl.ts                 # Firecrawl API wrapper
│   │
│   └── integrations/supabase/
│       ├── client.ts                    # Supabase client
│       └── types.ts                     # DB types (auto-gen)
│
├── supabase/functions/
```

```
│   ├── scrape-district/index.ts          # Multi-phase district scraper
│   ├── generate-email/index.ts           # AI email generation
│   ├── firecrawl-scrape/index.ts         # Single URL scrape
│   └── firecrawl-map/index.ts            # Sitemap discovery
│
└── supabase/config.toml
```

Database Schema

`florida_districts` - 67 County Districts
```sql
CREATE TABLE florida_districts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  county TEXT NOT NULL,
  district_name TEXT NOT NULL,
  district_code TEXT,
  website_url TEXT,
  student_count INTEGER,
  el_student_count INTEGER,
  el_percentage NUMERIC,
  scrape_status TEXT DEFAULT 'pending',  -- pending, in_progress, complete, error
  scrape_phase TEXT,                     -- mapping, contacts, research
  last_scraped_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);
```

`district_contacts` - Scraped Contacts
```sql
CREATE TABLE district_contacts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  district_id UUID NOT NULL REFERENCES florida_districts(id),
  full_name TEXT NOT NULL,
  first_name TEXT,
  last_name TEXT,
  email TEXT,
  title TEXT,
  department TEXT,
  phone TEXT,
  source_url TEXT,
```

```
  confidence_score NUMERIC DEFAULT 0.5,
  verified_at TIMESTAMPTZ,
  is_primary_contact BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);
```

`district_research` - Scraped Content
```sql
CREATE TABLE district_research (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  district_id UUID NOT NULL REFERENCES florida_districts(id),
  research_type TEXT NOT NULL,  -- strategic_plan, budget, news, el_program, url_mapping
  title TEXT,
  content TEXT,
  source_url TEXT,
  source TEXT DEFAULT 'firecrawl',  -- firecrawl, perplexity
  extracted_priorities JSONB,
  extracted_goals JSONB,
  file_path TEXT,
  scraped_at TIMESTAMPTZ DEFAULT now(),
  created_at TIMESTAMPTZ DEFAULT now()
);
```

`district_emails` - Generated Emails
```sql
CREATE TABLE district_emails (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  district_id UUID NOT NULL REFERENCES florida_districts(id),
  contact_id UUID REFERENCES district_contacts(id),
  subject_line TEXT NOT NULL,
  email_body TEXT NOT NULL,
  personalization_notes TEXT,
  generation_context JSONB,
  status TEXT DEFAULT 'draft',  -- draft, approved, exported
  approved_by UUID,
  approved_at TIMESTAMPTZ,
  exported_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);
```

`user_roles` - Admin Access
```sql
CREATE TABLE user_roles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL,
  role app_role NOT NULL,  -- 'admin' | 'user'
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TYPE app_role AS ENUM ('admin', 'user');

CREATE FUNCTION has_role(_role app_role, _user_id UUID) RETURNS BOOLEAN AS $$
  SELECT EXISTS (
    SELECT 1 FROM user_roles WHERE user_id = _user_id AND role = _role
  )
$$ LANGUAGE sql SECURITY DEFINER;
```

Edge Functions Architecture

1. `scrape-district` - Multi-Phase District Scraper

File: `supabase/functions/scrape-district/index.ts`

Architecture: Uses `EdgeRuntime.waitUntil()` for background processing (avoids 400s timeout).

Three Phases:

Phase	Action	Output
1. Mapping	Firecrawl maps district website	`district_research` (type: url_mapping)
2. Contacts	Extract contacts from key pages	`district_contacts` with confidence scores
3. Research	Scrape strategic content + Perplexity deep research	`district_research` (types: strategic_plan, news, el_program)

Key Implementation Details:

```typescript
// Phase 1: Map website
const mapResult = await fetch('https://api.firecrawl.dev/v1/map', {
```

```
  method: 'POST',
  headers: { Authorization: `Bearer ${firecrawlApiKey}` },
  body: JSON.stringify({ url: websiteUrl, limit: 5000 })
});

// Identify key pages using GPT
const keyPages = await identifyKeyPages(urls);  // leadership, strategic_plan, esol, budget

// Save URL mapping for Phase 2
await supabase.from('district_research').delete()
  .eq('district_id', districtId)
  .eq('research_type', 'url_mapping');
await supabase.from('district_research').insert({
  district_id: districtId,
  research_type: 'url_mapping',
  content: JSON.stringify(keyPages),
  source: 'firecrawl'
});

// Chain to Phase 2
EdgeRuntime.waitUntil(runPhase2Contacts(districtId, supabase, firecrawlApiKey));
```

Phase 2 Contact Extraction:
```typescript
// Read URL mapping from Phase 1
const { data: urlMapping } = await supabase.from('district_research')
  .select('content')
  .eq('district_id', districtId)
  .eq('research_type', 'url_mapping')
  .single();

// Scrape leadership pages
for (const url of keyPages.leadership) {
  const scraped = await firecrawlScrape(url);
  const contacts = await extractContactsWithGPT(scraped.markdown);
  // Filter by EL-relevance, calculate confidence
  await saveContacts(districtId, contacts);
}
```

Phase 3 Research:
```typescript
// Firecrawl: strategic plans, budgets
```

```
for (const url of keyPages.strategic_plan) {
  const scraped = await firecrawlScrape(url);
  const extracted = await extractELInitiatives(scraped.markdown);
  await supabase.from('district_research').insert({
    district_id: districtId,
    research_type: 'strategic_plan',
    content: scraped.markdown,
    extracted_priorities: extracted.priorities,
    source: 'firecrawl'
  });
}

// Perplexity: current news
const newsQuery = `${districtName} Florida English learners ESOL funding programs
2024-2025`;
const news = await perplexitySearch(newsQuery, 'sonar-deep-research');
await supabase.from('district_research').insert({
  district_id: districtId,
  research_type: 'news',
  content: news.content,
  source: 'perplexity'
});
```

---

2. `generate-email` - AI Email Generator

File: `supabase/functions/generate-email/index.ts`

Flow:
1. Receive: contact info, district context, service selection, state
2. **Parallel research**: Perplexity (funding + compliance) + Knowledge base search
3. **Rerank** documents with GPT-4o for relevance
4. **Score case studies** (only include if relevance ≥70)
5. **Build prompt** with all context
6. **Generate email** via SSE streaming (GPT-5)

Key Code Patterns:

```typescript
// Parallel research execution
const [fundingResults, complianceResults, kbResults] = await Promise.all([
  perplexitySearch(`${state} English learner funding sources schools 2025`, 'sonar-pro'),
```

```
  perplexitySearch(`${state} English learner compliance requirements mandates`, 'sonar-pro'),
  supabase.rpc('search_knowledge_hybrid', { query_embedding, query_text, match_count: 10 })
]);

// Rerank with GPT-4o
const reranked = await rerankDocuments(kbResults, serviceSelection);

// Score case studies (70+ threshold)
const caseStudies = await scoreCaseStudies(reranked, serviceSelection);
const relevantCaseStudies = caseStudies.filter(cs => cs.score >= 70);

// Build system prompt
const systemPrompt = `
You are writing a sales email for WCEPS.

RECIPIENT:
- Name: ${contact.full_name}
- Title: ${contact.title}
- District: ${districtName}

DISTRICT CONTEXT:
${districtResearch.map(r => r.content).join('\n')}

FUNDING OPPORTUNITIES:
${fundingResults.content}

COMPLIANCE REQUIREMENTS:
${complianceResults.content}

${relevantCaseStudies.length > 0 ? `CASE STUDIES:\n${relevantCaseStudies.map(cs =>
cs.content).join('\n')}` : ''}

STYLE GUIDE:
- 150-180 words
- 2-3 short paragraphs + 2-3 bullets
- Subject line under 50 characters
- Opening hook under 12 words
- Active voice, plain text only
- NO markdown, NO citations
- Integrate ONE funding source or compliance fact into body
`;

// Stream response
const response = await fetch('https://lovable.dev/api/v1/chat', {
```

```
  method: 'POST',
  headers: { Authorization: `Bearer ${lovableApiKey}` },
  body: JSON.stringify({
    model: 'openai/gpt-5',
    messages: [{ role: 'system', content: systemPrompt }, { role: 'user', content: 'Generate the email.' }],
    stream: true
  })
});
```

SSE Streaming Pattern (keeps connection alive during long research):
```typescript
const encoder = new TextEncoder();
const stream = new ReadableStream({
  async start(controller) {
    // Send progress events
    controller.enqueue(encoder.encode(`data: {"type":"progress","message":"Researching funding..."}\n\n`));

    const funding = await perplexitySearch(...);
    controller.enqueue(encoder.encode(`data: {"type":"progress","message":"Generating email..."}\n\n`));

    // Stream final email
    for await (const chunk of emailStream) {
      controller.enqueue(encoder.encode(`data: {"type":"content","text":"${chunk}"}\n\n`));
    }
    controller.enqueue(encoder.encode(`data: {"type":"done"}\n\n`));
    controller.close();
  }
});

return new Response(stream, {
  headers: { ...corsHeaders, 'Content-Type': 'text/event-stream' }
});
```

---

3. `firecrawl-scrape` & `firecrawl-map`

Wrapper functions for Firecrawl API:

```typescript
// firecrawl-scrape/index.ts
const response = await fetch('https://api.firecrawl.dev/v1/scrape', {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${Deno.env.get('FIRECRAWL_API_KEY')}`,
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    url,
    formats: ['markdown'],
    onlyMainContent: true
  })
});

// firecrawl-map/index.ts
const response = await fetch('https://api.firecrawl.dev/v1/map', {
  method: 'POST',
  headers: { 'Authorization': `Bearer ${Deno.env.get('FIRECRAWL_API_KEY')}` },
  body: JSON.stringify({ url, limit: 5000 })
});
```

Frontend Components

DistrictEmails.tsx (`/email/districts`)

Main UI for bulk email generation:

```typescript
// Fetch districts with contacts/research counts
const { data: districts } = useQuery({
  queryKey: ['florida-districts'],
  queryFn: async () => {
    const { data } = await supabase
      .from('florida_districts')
      .select(`
        *,
        district_contacts(count),
        district_research(count),
        district_emails(count)
      `)
```

```
      .order('county');
    return data;
  }
});

// Trigger scrape
const handleScrape = async (districtId: string) => {
  await supabase.functions.invoke('scrape-district', {
    body: { districtId, phase: 1 }
  });
};

// Progress bar maps scrape_phase to percentage
const getProgress = (phase: string) => {
  switch (phase) {
    case 'mapping': return 20;
    case 'contacts': return 50;
    case 'research': return 80;
    default: return 100;
  }
};
```

EmailGenerator.tsx (`/email`)

Single email generation with form:

```typescript
const [formData, setFormData] = useState({
  contactName: '',
  contactTitle: '',
  contactEmail: '',
  districtName: '',
  state: 'Florida',
  services: [],  // multi-select
  audiences: [], // multi-select
  customInstructions: ''
});

// SSE streaming
const generateEmail = async () => {
  const response = await fetch(`${supabaseUrl}/functions/v1/generate-email`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
```

```
    body: JSON.stringify(formData)
  });

  const reader = response.body.getReader();
  while (true) {
    const { done, value } = await reader.read();
    if (done) break;
    const text = new TextDecoder().decode(value);
    // Parse SSE events and update UI
  }
};
```

Key Business Logic

Contact Targeting

Three-layer filtering system:

1. **Expanded terminology**: Matches titles containing:
   - "Multilingual", "Language Acquisition", "ESOL", "ELL", "Bilingual"
   - "Curriculum", "Instruction", "Professional Development"
   - "Superintendent", "Assistant Superintendent"

2. **Aggressive exclusion**: Rejects contacts in:
   - IT, Technology, Procurement, Finance, Transportation
   - Maintenance, Food Service, Athletics, Security

3. **Confidence scoring** (0-1 scale):
   - Title match strength: +0.3
   - Department relevance: +0.2
   - Perplexity verification: +0.3
   - Email found: +0.2

Only contacts with **≥80% confidence** are stored.

Email Style Guide (Hardcoded)

```
STRUCTURE:
- 150-180 words total
- 2-3 short paragraphs
```

- 2-3 bullets
- Subject line under 50 characters

OPENING:
- First sentence: ONE short question or statement (UNDER 12 WORDS)
- About recipient's pain point
- Active voice only

CONTENT:
- Integrate ONE funding source OR compliance fact
- Include WIDA MODEL bullet: "Only interim assessment that predicts ACCESS scores"
- NO deficit language - frame as "opportunities" not burdens
- Plain text only - NO markdown, NO citations
- NO wisc.edu URLs - only wceps.org and Calendly

CASE STUDIES:
- Only include if relevance score ≥70
- Omit entirely if no relevant examples
```

### Salesforce Export Format

CSV with columns:
```
First Name, Last Name, Email, Title, Company, Subject Line, Email Body
```

---

## Frontend Routes

| Route | Page | Purpose |
|---|---|---|
| `/` | Index.tsx | Landing |
| `/auth` | Auth.tsx | Login/signup |
| `/email` | EmailGenerator.tsx | Single email generator |
| `/email/districts` | DistrictEmails.tsx | Bulk FL district emails |

---

## RLS Policies

| Table | Public Read | Admin Write |
|---|---|---|
```

`florida_districts`	✅	✅
`district_contacts`	✅	✅
`district_research`	✅	✅
`district_emails`	❌	✅
`user_roles`	Own only	✅

Testing

Test District Scrape
1. Navigate to `/email/districts`
2. Click "Scrape" on any district
3. Watch progress bar: Mapping (20%) → Contacts (50%) → Research (80%) → Complete (100%)
4. Check logs in Supabase Dashboard → Edge Functions

Test Email Generation
1. Navigate to `/email`
2. Fill form with contact info, select services
3. Click Generate
4. Watch SSE progress updates
5. Review generated email

Troubleshooting

"No URL mapping data found from Phase 1"
- Phase 1 uses delete-then-insert pattern
- Check `district_research` table for `research_type = 'url_mapping'`
- Verify Phase 1 completed successfully in logs

Scrape stuck in "in_progress"
- Check Edge Function logs for errors
- Reset: `UPDATE florida_districts SET scrape_status = 'pending', scrape_phase = NULL WHERE id = '...'`

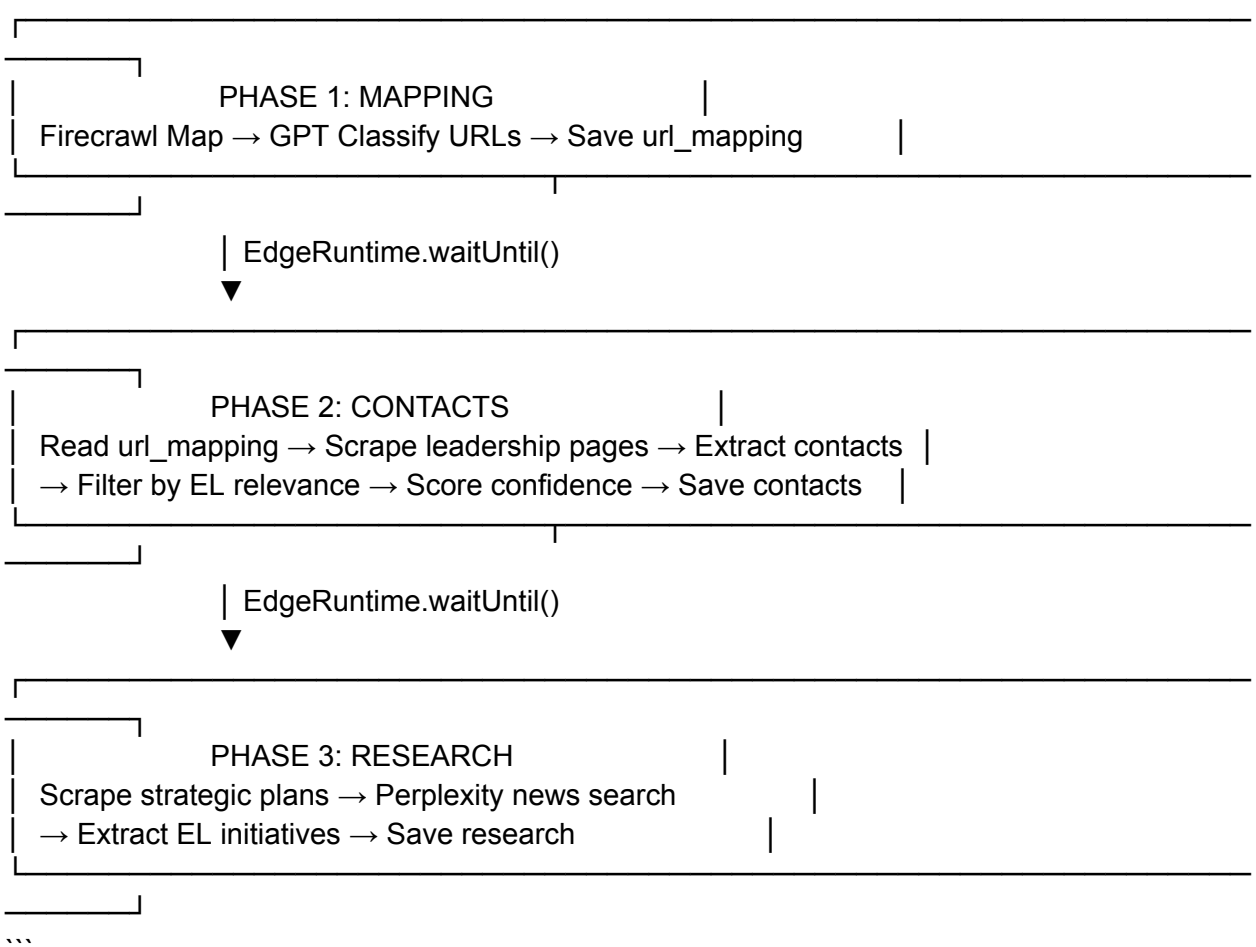
Email generation timeout
- SSE streaming should prevent this
- Check Perplexity API key validity
- Verify LOVABLE_API_KEY is set

No contacts extracted

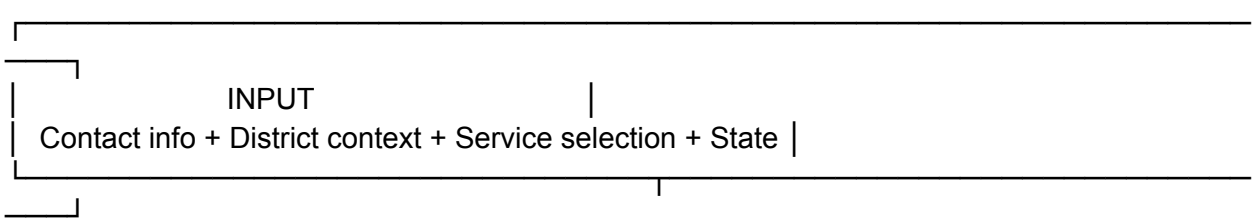
- Check if leadership pages were identified in Phase 1
- Review Firecrawl response in logs
- Verify contact extraction GPT prompt

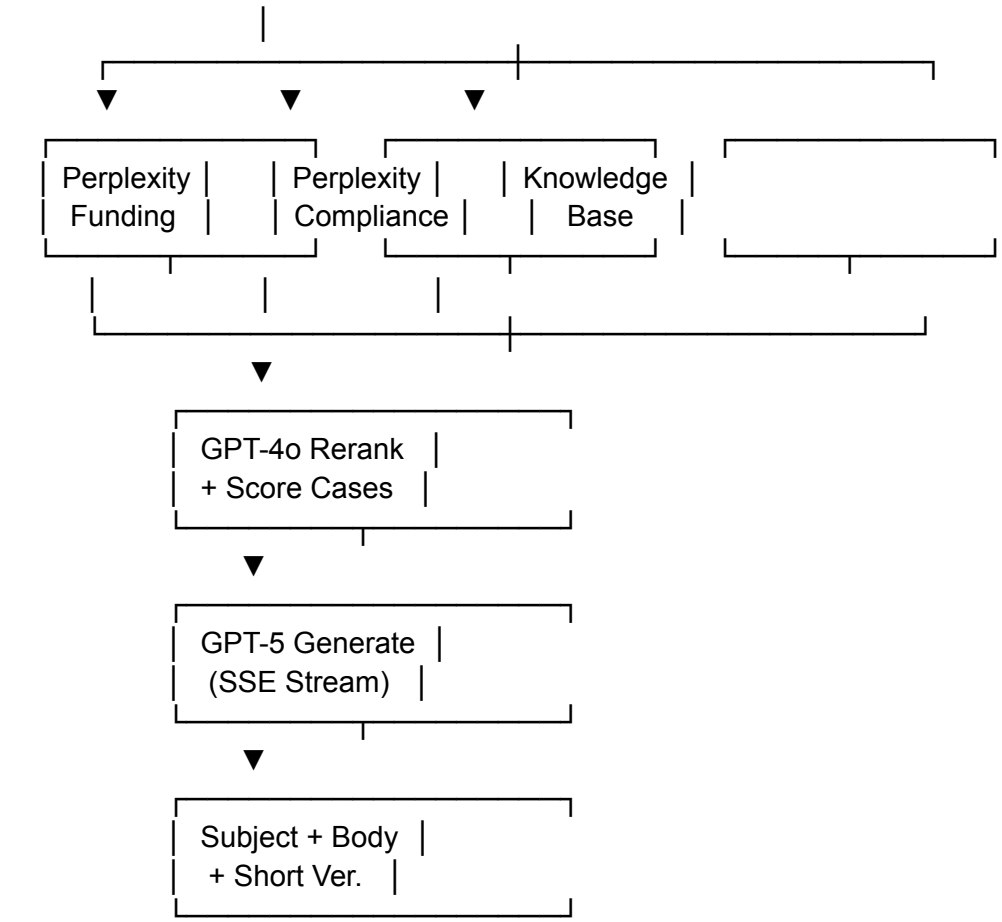
Architecture Diagrams

District Scraping Flow



Email Generation Flow






```

---

## Quick Reference Commands

```bash
# Local development
npm run dev

# Deploy edge functions (if using Supabase CLI)
supabase functions deploy scrape-district
supabase functions deploy generate-email
supabase functions deploy firecrawl-scrape
supabase functions deploy firecrawl-map

# Reset a stuck scrape
UPDATE florida_districts
SET scrape_status = 'pending', scrape_phase = NULL
WHERE county = 'Osceola';
```
```

```
```